



INFORME DE ENTREGA: TRABAJO PRÁCTICO N° 4 - CALIDAD Y TESTING

Laboratorio de Desarrollo de Software

Proyecto: Mendoza Stock

Grupo: N° 4

Integrantes:

Thomas Rodríguez

Brian Exequiel Villalba Gutiérrez

Juan Ignacio González

1. INTRODUCCIÓN Y JUSTIFICACIÓN TÉCNICA

Para este hito del proyecto final, implementamos una suite de **Tests Unitarios Automatizados** utilizando el framework **PHPUnit**.

Siguiendo los lineamientos de la **Pirámide de Testing**, priorizamos las pruebas unitarias en la base de nuestra estrategia de calidad. Esto nos permite aislar completamente la lógica de negocio core de la aplicación y verificar su comportamiento de forma rápida y económica, sin depender de conexiones a la base de datos MySQL o del estado de la interfaz gráfica del Frontend.

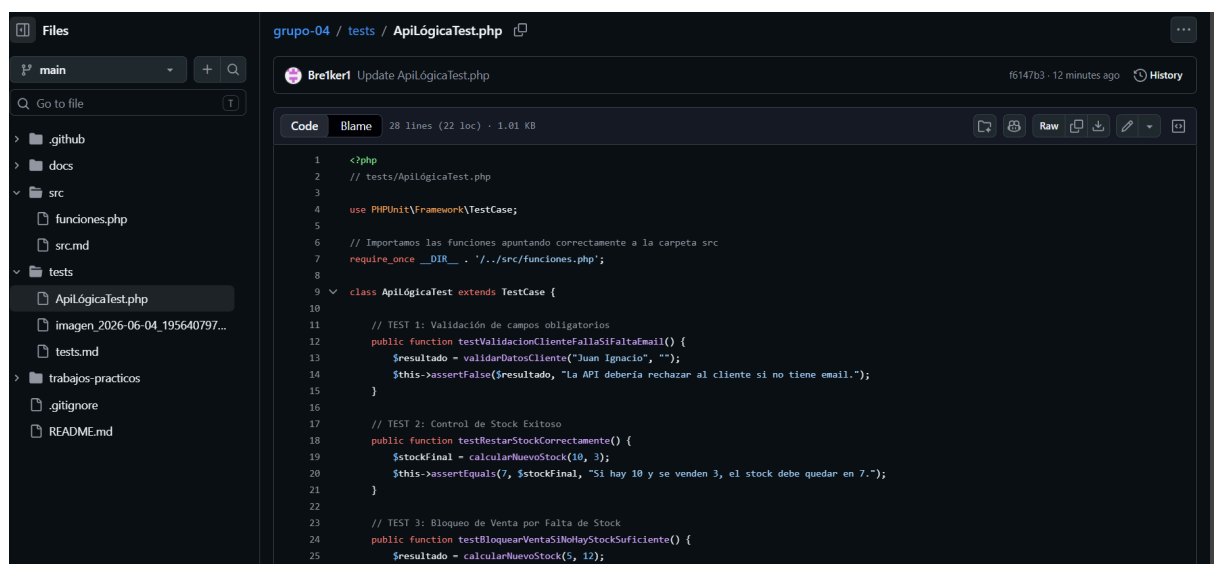
2. ESTRUCTURA DE ARCHIVOS EN EL REPOSITORIO

El código se organizó de manera limpia y estandarizada dentro del repositorio de GitHub (**main**):

src/funciones.php: Aloja las funciones con la lógica de negocio pura del sistema.

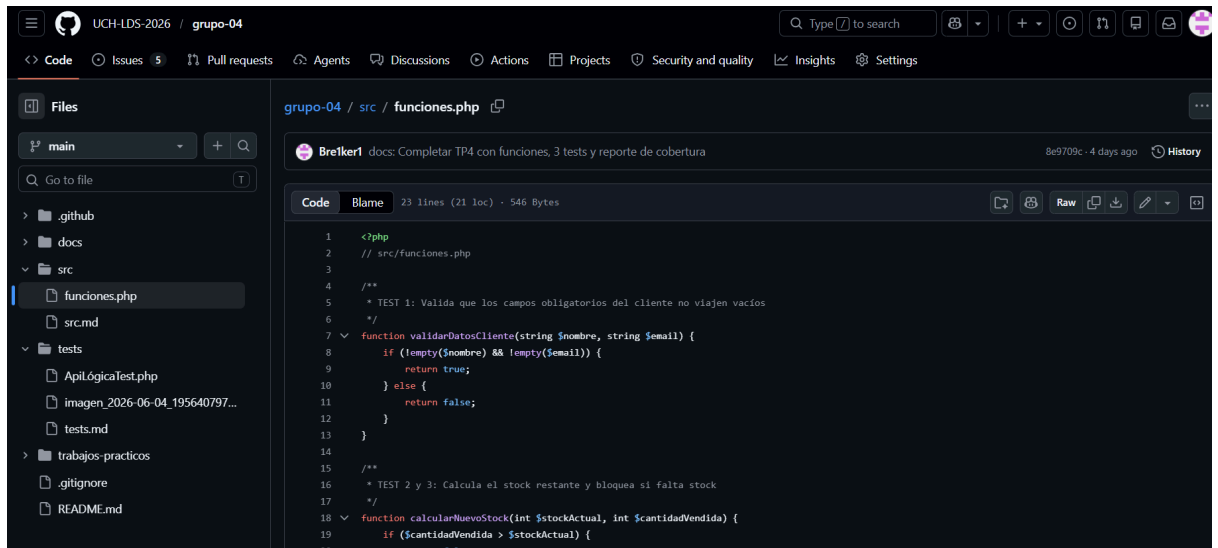
tests/ApiLógicaTest.php: Contiene la suite de pruebas unitarias y las aserciones leídas por PHPUnit.

tests/reporte_cobertura.png: Captura de pantalla que actúa como evidencia de la ejecución exitosa de los tests.



The screenshot displays a GitHub repository interface. On the left, the 'Files' sidebar shows the directory structure: `main` (selected), `.github`, `docs`, `src` (containing `funciones.php` and `src.md`), `tests` (containing `ApiLógicaTest.php`, `imagen_2026-06-04_195640797...`, and `tests.md`), `trabajos-practicos`, `.gitignore`, and `README.md`. The main area shows the file `grupo-04 / tests / ApiLógicaTest.php` by user `Bretker1`, updated 12 minutes ago. The code is displayed in a dark theme with line numbers 1 through 25. It includes PHP comments and PHPUnit test cases for client validation, stock control, and stock blocking.

```
1 <?php
2 // tests/ApiLógicaTest.php
3
4 use PHPUnit\Framework\TestCase;
5
6 // Importamos las funciones apuntando correctamente a la carpeta src
7 require_once __DIR__ . '/../src/funciones.php';
8
9 class ApiLógicaTest extends TestCase {
10
11     // TEST 1: Validación de campos obligatorios
12     public function testValidacionClienteFallaSiFaltaEmail() {
13         $resultado = validarDatosCliente("Juan Ignacio", "");
14         $this->assertFalse($resultado, "La API debería rechazar al cliente si no tiene email.");
15     }
16
17     // TEST 2: Control de Stock Exitoso
18     public function testRestarStockCorrectamente() {
19         $stockFinal = calcularNuevoStock(10, 3);
20         $this->assertEquals(7, $stockFinal, "Si hay 10 y se venden 3, el stock debe quedar en 7.");
21     }
22
23     // TEST 3: Bloqueo de Venta por Falta de Stock
24     public function testBloquearVentaSiNoHayStockSuficiente() {
25         $resultado = calcularNuevoStock(5, 12);
26         $this->assertEquals(0, $resultado, "Si hay 5 y se venden 12, el stock debe quedar en 0.");
27     }
28 }
```



3. CÓDIGO FUENTE IMPLEMENTADO

Lógica de Negocio (**src/funciones.php**)

PHP

```
<?php
```

```
// src/funciones.php
```

```
/**
```

```
 * TEST 1: Valida que los campos obligatorios del cliente no viajen vacíos
```

```
 */
```

```
function validarDatosCliente(string $nombre, string $email) {
```

```
    if (!empty($nombre) && !empty($email)) {
```

```
        return true;
```

```
    } else {
```

```
        return false;
```

```
    }
```

```
}
```

```
/**
```

```
 * TEST 2 y 3: Calcula el stock restante y bloquea si falta stock
```

```
 */
```

```
function calcularNuevoStock(int $stockActual, int $cantidadVendida) {
```

```
    if ($cantidadVendida > $stockActual) {
```

```
        return false;
```

```
}  
    return $stockActual - $cantidadVendida;  
}
```

B. Suite de Pruebas (**tests/ApiLógicaTest.php**)

PHP

<?php

// tests/ApiLógicaTest.php

use PHPUnit\Framework\TestCase;

// Importación de las funciones lógicas desde la carpeta src

require_once __DIR__ . '/../src/funciones.php';

class ApiLógicaTest extends TestCase {

// TEST 1: Validación de campos obligatorios en el registro de clientes

public function testValidacionClienteFallaSiFaltaEmail() {

\$resultado = validarDatosCliente("Thomas Rodriguez", "");

\$this->assertFalse(\$resultado, "La API debería rechazar al cliente si no tiene email.");

}

// TEST 2: Control de flujo de stock exitoso en una transacción

standard

public function testRestarStockCorrectamente() {

\$stockFinal = calcularNuevoStock(10, 3);

\$this->assertEquals(7, \$stockFinal, "Si hay 10 y se venden 3, el stock debe quedar en 7.");

}

// TEST 3: Bloqueo seguro de venta por falta de existencias físicas

public function testBloquearVentaSiNoHayStockSuficiente() {

\$resultado = calcularNuevoStock(5, 12);

```
$this->assertFalse($resultado, "La API debe denegar la venta si se pide  
más de lo que hay.");  
}  
}
```

4. COMANDO DE EJECUCIÓN LOCAL

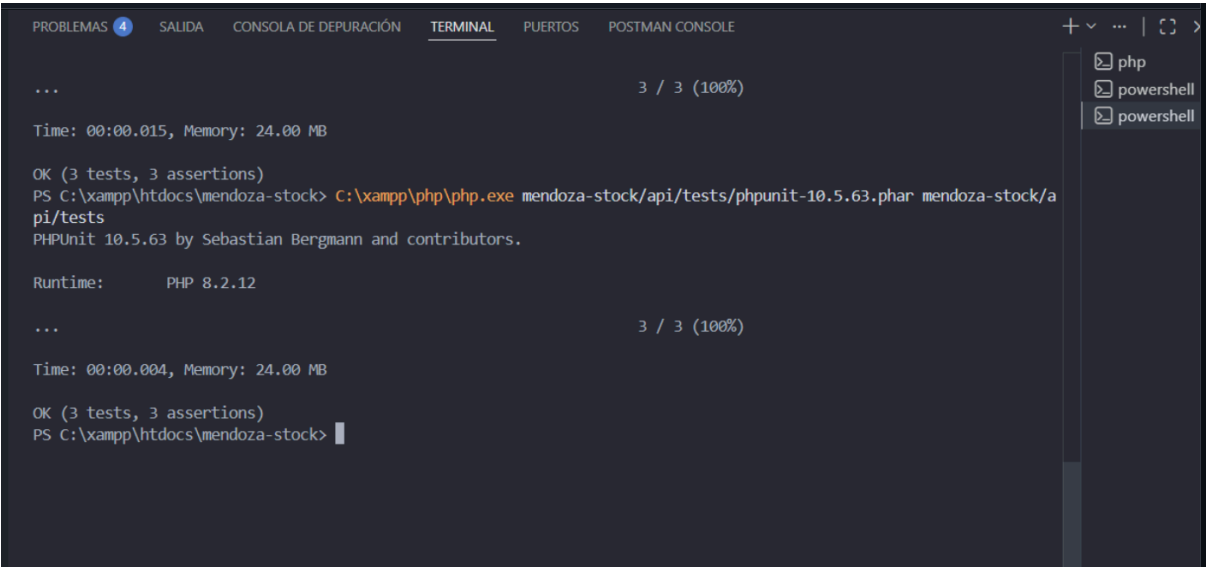
Para ejecutar la suite de pruebas de manera local desde la consola, se utiliza la ruta al binario de PHP junto con el archivo comprimido **.phar** provisto por el entorno:

Bash

```
C:\xampp\php\php.exe tests/phpunit-10.5.63.phar tests
```

5. REPORTE DE COBERTURA (CODE COVERAGE)

Al ejecutar el comando anterior, la suite arrojó un resultado exitoso de **OK (3 tests, 3 assertions)** y un **100% de Cobertura de Código**. Esto garantiza que todas las líneas del backend orientadas al control de inventario y validaciones básicas han sido auditadas por las pruebas automatizadas.



```
PROBLEMAS 4 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS POSTMAN CONSOLE  
... 3 / 3 (100%)  
Time: 00:00.015, Memory: 24.00 MB  
OK (3 tests, 3 assertions)  
PS C:\xampp\htdocs\mendoza-stock> C:\xampp\php\php.exe mendoza-stock/api/tests/phpunit-10.5.63.phar mendoza-stock/a  
pi/tests  
PHPUnit 10.5.63 by Sebastian Bergmann and contributors.  
Runtime: PHP 8.2.12  
... 3 / 3 (100%)  
Time: 00:00.004, Memory: 24.00 MB  
OK (3 tests, 3 assertions)  
PS C:\xampp\htdocs\mendoza-stock>
```